

Plataforma de Simulación de Pruebas de Fondeo (Prop Firm Challenge Tracker)

Hoja de ruta técnica para el equipo de desarrollo

Documento de arquitectura, alcance funcional, modelo de datos y plan de entregas

Versión 1.0 · Preparado para TKECH · Septiembre 2026

1. Resumen del proyecto

Se construirá una plataforma web donde el trader elige el tipo de prueba de fondeo (**Forex, Futuros o Cripto**), selecciona la firma prop y el tamaño de cuenta, y el sistema aplica automáticamente el preset de reglas correspondiente (objetivo de ganancia, drawdown diario, drawdown máximo, días mínimos, etc.). Para Forex y Futuros, el usuario abre una cuenta demo en su bróker y descarga un **Expert Advisor (EA) propio de reporte**, que adjunta a un gráfico de esa cuenta e identifica con una clave única generada por la plataforma. Para Cripto, el usuario conecta una cuenta de exchange (testnet o de evaluación) mediante una **API key de solo lectura, sin permiso de retiro**. En ambos casos, la plataforma calcula en tiempo real los parámetros de la prueba, muestra un dashboard y envía notificaciones automáticas cuando el usuario aprueba la prueba, la pierde, o toca el límite de drawdown diario.

Objetivos clave

- Simular con precisión las reglas reales de cada prop firm, sin depender de código fijo por firma (presets configurables desde un panel admin).
- Reportar los datos de la cuenta directamente desde el terminal del usuario (MT4 / MT5 / cTrader) mediante un EA/cBot propio, sin manejar ni almacenar contraseñas del bróker.
- Soportar también pruebas sobre **exchanges cripto** mediante API keys de solo lectura, con el mismo motor de reglas usado para Forex y Futuros.
- Calcular en tiempo real: balance, equity, PnL diario, drawdown diario, drawdown máximo, días operados y progreso hacia el objetivo.
- Notificar automáticamente eventos críticos: aprobación, reprobación, alerta de DD diario cercano al límite.
- Base de datos y backend sobre Supabase, con posibilidad de escalar a más firmas y activos en el futuro.

2. Alcance funcional

2.1 Flujo de usuario (MVP)

- **Paso 1 — Tipo de prueba:** el usuario elige Forex, Futuros o Cripto.
- **Paso 2 — Firma prop:** según el tipo elegido, se listan las firmas activas (ej. Forex: FTMO, Bullfy, FX Live Capital, Upcomers, FundedNext, The5ers, FundingPips · Futuros: Apex, Lucid, Upcomers, Topstep, MyFundedFutures · Cripto: firmas a definir y cargar vía el panel admin, ya que el listado inicial del cliente no especificó nombres para esta categoría).
- **Paso 3 — Preset de cuenta:** el usuario elige el tamaño/plan de cuenta de esa firma (ej. 10K, 25K, 50K, 100K, 1 o 2 fases, etc.) y el sistema carga automáticamente las reglas asociadas.
- **Paso 4 — Vinculación de cuenta:** el método de vinculación depende del mercado elegido: para **Forex/Futuros**, la plataforma genera una clave única de reporte y el usuario adjunta el EA (MT4/MT5) o cBot (cTrader) a su cuenta demo con esa clave como parámetro; para **Cripto**, el usuario genera en el exchange una **API key de solo lectura, sin permiso de retiro ni de transferencia**, y la pega en la plataforma.
- **Paso 5 — Dashboard en vivo:** tarjetas y gráficos con balance, equity, PnL del día, drawdown diario vs. límite, drawdown máximo vs. límite, objetivo de ganancia vs. progreso, días operados vs. mínimo requerido, y estado general (En curso / Aprobado / Reprobado).
- **Paso 6 — Notificaciones:** email (y opcionalmente push/WhatsApp) cuando: se aprueba la prueba, se reprueba (violación de regla), o se alcanza un umbral configurable del drawdown diario (ej. 80% del límite).

2.2 Panel de administración (importante para escalabilidad)

Para no depender de despliegues de código cada vez que una firma cambie sus reglas o se agregue una nueva, se recomienda construir un panel admin donde se gestionen como datos (no como código):

- CRUD de firmas (nombre, categoría Forex/Futuros, logo, activo/inactivo).
- CRUD de presets por firma: tamaño de cuenta, fases, objetivo de ganancia por fase, drawdown diario, drawdown máximo (estático o trailing), días mínimos de trading, reglas especiales (ej. consistencia, noticias, fin de semana, EAs permitidos).
- Historial de cambios de reglas (por si una firma actualiza sus condiciones, se versiona el preset).

2.3 Fuera de alcance del MVP (fase 2+)

- Pagos / suscripciones dentro de la plataforma.
- Cuentas reales (live) — el MVP se limita a cuentas demo con investor password.
- App móvil nativa (se puede cubrir con un dashboard web responsive / PWA).
- Integración directa con TradeLocker / DXtrade / Match-Trader (se evalúa según qué brókers usen las firmas objetivo).

3. Arquitectura general

La arquitectura se divide en 4 bloques: (1) Frontend web, (2) Backend / base de datos en Supabase, (3) un servicio worker independiente encargado de conectarse a las cuentas de trading y calcular métricas, y (4) el motor de notificaciones. Se separa el worker del backend de Supabase porque las conexiones persistentes a MT4/MT5 (streaming en tiempo real) no encajan bien dentro de Edge Functions (que son de ejecución corta).

Bloque	Función	Tecnología sugerida
Frontend web	UI de selección de prueba, dashboard, panel admin	Next.js 14 (React) + TypeScript + TailwindCSS + shadcn/ui
Backend / datos	Auth, base de datos, reglas de acceso, funciones ligeras	Supabase (Postgres + Auth + Row Level Security + Realtime + Edge Functions)
Servicio receptor + motor de reglas	Recibe los reportes del EA/cBot propio, valida, calcula métricas y aplica el preset de reglas	Node.js (NestJS o Fastify) como endpoint webhook, desplegado en Railway / Fly.io / Render
EA / cBot de reporte (propio)	Se ejecuta en el terminal del usuario, lee balance/equity/operaciones y las envía por HTTP	MQL5 (MT5), MQL4 (MT4) y C#/cAlgo (cTrader) — desarrollo interno, sin depender de terceros
Conector de exchanges cripto	Se conecta vía API oficial del exchange (Binance, Bybit, etc.) con una key de solo lectura para leer balance, posiciones e historial	Node.js + librería CCXT (unifica la API de múltiples exchanges), integrado en el mismo servicio receptor
Notificaciones	Envío de correos y alertas en tiempo real	Resend o SendGrid (email) + Supabase Realtime (in-app) + OneSignal (push, opcional)

Flujo de datos — Forex/Futuros (push): el EA/cBot instalado en el terminal del usuario lee cada cierto intervalo (ej. cada 15–60 segundos vía OnTimer) el balance, equity, posiciones abiertas y operaciones cerradas recientes → arma un JSON y lo envía por HTTPS (WebRequest en MQL / HttpClient en cAlgo) al endpoint receptor, identificándose con la clave única del desafío.

Flujo de datos — Cripto (pull): el backend consulta periódicamente la API oficial del exchange usando la API key de solo lectura del usuario (ej. cada 30–60 segundos, o vía WebSocket si el exchange lo soporta) para obtener balance, posiciones abiertas e historial de operaciones.

En ambos casos, el backend calcula drawdown diario, drawdown máximo y progreso hacia el objetivo aplicando el preset de la firma → guarda un snapshot en Supabase → si se cruza una regla (aprobación, violación, alerta de DD), dispara un evento que activa la notificación correspondiente.

4. Stack tecnológico específico

Capa	Tecnología	Notas
Frontend	Next.js 14 + TypeScript + TailwindCSS + shadcn/ui + Recharts	SSR/SSG para landing, CSR para dashboard en vivo
Hosting frontend	Vercel	Despliegue continuo desde GitHub
Base de datos	Supabase (PostgreSQL)	Incluye Auth, Storage y Realtime

Capa	Tecnología	Notas
Autenticación	Supabase Auth	Email/contraseña + Google OAuth
Seguridad de datos	Row Level Security (RLS) + claves de reporte con hash (no se manejan contraseñas de bróker)	El sistema nunca ve ni almacena credenciales del bróker del usuario
EA / cBot de reporte	MQL5 (MT5) + MQL4 (MT4) + C#/cAlgo (cTrader), desarrollo propio	Ver sección 6 — reemplaza a un proveedor externo de conectividad
Endpoint receptor de reportes	Node.js (Fastify/NestJS) o Supabase Edge Function	Recibe el POST del EA, valida la clave y el origen, calcula reglas y guarda snapshot
Conector cripto	CCXT (librería Node.js/Python que unifica APIs de Binance, Bybit, OKX, etc.)	Se usa con API keys de solo lectura del usuario, sin permisos de retiro
Hosting del receptor	Railway, Fly.io, Render o Supabase Edge Functions	Puede ser serverless al ser una API que solo recibe reportes (no mantiene conexiones persistentes salientes)
Cola de tareas / jobs	Supabase Cron / pg_cron o BullMQ + Redis	Para verificaciones periódicas (ej. reseteo diario a medianoche del bróker)
Notificaciones email	Resend o SendGrid	Plantillas: aprobado / reprobado / alerta DD
Notificaciones in-app	Supabase Realtime (WebSockets)	Actualiza el dashboard sin refrescar
Notificaciones push (opcional)	OneSignal	Fase 2
Monitoreo / logs	Sentry (errores) + Better Stack / Logtail (logs)	Crítico en el worker por ser el punto de falla más sensible
CI/CD	GitHub Actions	Lint, tests y despliegue automático
Testing	Vitest / Jest (unitario) + Playwright (E2E)	Probar sobre todo el motor de cálculo de reglas

5. Modelo de datos (Supabase / PostgreSQL)

Estructura relacional propuesta para el MVP. Los nombres son sugeridos; el equipo puede ajustarlos al convenio interno.

profiles — Extiende auth.users de Supabase.

id (uuid, FK a auth.users), full_name, email, created_at

firms — Firmas prop disponibles.

id, name, category (forex/futures/crypto), logo_url, is_active, created_at

firm_presets — Reglas por firma y tamaño de cuenta (versionadas).

id, firm_id (FK), account_size, phase_count, profit_target_phase1, profit_target_phase2, max_daily_dd_pct, max_total_dd_type (static/trailing), max_total_dd_pct, min_trading_days, extra_rules (jsonb), version, is_active

challenges — Instancia de una prueba iniciada por un usuario.

id, user_id (FK), firm_id (FK), preset_id (FK), platform (mt4/mt5/ctrader/exchange), broker_name o exchange_name, account_login (solo referencia), report_key_hash (para el flujo EA, con hash), exchange_api_key_encrypted / exchange_api_secret_encrypted (solo para el flujo cripto, cifradas), status (active/passed/failed/breached), current_phase, start_date, high_water_mark, initial_balance, last_report_at, connection_status (connected/stale/disconnected)

account_snapshots — Fotografías periódicas de la cuenta (para gráficos e historial).

id, challenge_id (FK), captured_at, balance, equity, daily_pl, current_daily_dd_pct, current_total_dd_pct, open_positions

trades — Historial de operaciones sincronizadas desde el bróker.

id, challenge_id (FK), ticket, symbol, side, volume, open_time, close_time, open_price, close_price, profit, commission, swap

rule_events — Eventos disparados por el motor de reglas.

id, challenge_id (FK), event_type (target_reached/daily_dd_warning/daily_dd_breach/max_dd_breach/passed/failed), details (jsonb), created_at

notifications — Registro de notificaciones enviadas.

id, user_id (FK), challenge_id (FK), channel (email/push/in_app), event_type, sent_at, status

Nota de seguridad: en el flujo **EA/cBot (Forex/Futuros)** nunca se maneja ni almacena una credencial del bróker — solo se guarda el hash de la clave de reporte (**report_key_hash**), igual que se haría con una contraseña de usuario. En el flujo **Cripto**, sí se recibe una credencial (la API key/secret del exchange), por lo que debe cifrarse en reposo (Supabase Vault o pgcrypto) y la plataforma debe validar/instruir al usuario para que la genere con permisos de **solo lectura, sin retiro ni transferencia** — la mayoría de exchanges permiten crear API keys con ese alcance restringido.

6. Módulo crítico: EA/cBot propio de reporte

Se descarta el uso de un proveedor externo (tipo MetaApi.cloud) y se construye un **Expert Advisor propio** (MQL4/MQL5) y un **cBot propio** (cAlgo/C#) que el usuario instala en su propia cuenta demo. El EA vive dentro del terminal del usuario, lee los datos directamente con las funciones nativas de la plataforma (sin necesidad de ninguna contraseña) y los envía por HTTPS al backend. Esta opción aprovecha la experiencia ya existente del equipo en desarrollo MQL5/cAlgo y elimina por completo la necesidad de manejar credenciales del bróker.

6.1 Funcionamiento del EA (MT4/MT5)

- El usuario descarga el archivo compilado (.ex5 / .ex4) desde la plataforma y lo adjunta a un gráfico de su cuenta demo.
- Como parámetro de entrada (input), pega la **clave única de reporte** generada por la plataforma al crear el desafío (vincula el EA con el registro correcto en Supabase).
- En **OnInit**: valida la clave contra el backend (ping inicial) y confirma que la cuenta es de tipo demo antes de continuar (usar `AccountInfoInteger(ACCOUNT_TRADE_MODE)`).
- En **OnTimer** (cada 15–60 segundos, configurable): lee `AccountInfoDouble(ACCOUNT_BALANCE/EQUITY/MARGIN)`, recorre posiciones abiertas y el historial de `HistoryDealsTotal` desde el último ticket ya reportado, arma un JSON y lo envía con `WebRequest()`.
- Reintentos con backoff si falla el envío (sin conexión, servidor caído) y cola local temporal en variables globales para no perder reportes.
- El usuario debe habilitar 'Algo Trading' y agregar el dominio del backend a la lista blanca en Herramientas → Opciones → Expert Advisors → 'Permitir WebRequest para las URL' — este paso debe explicarse con un wizard/tutorial dentro de la plataforma.

6.2 Funcionamiento del cBot (cTrader)

- Misma lógica que el EA, adaptada a la API de cAlgo (C#): lectura de `Account.Balance`, `Account.Equity`, `Positions` e `History`.
- El envío HTTP en cAlgo depende de la versión del sandbox de cTrader — debe validarse en Fase 0 si el entorno permite peticiones salientes directas desde un cBot; si está restringido, la alternativa es usar la **Open API oficial de cTrader** (OAuth) como método nativo para esa plataforma en lugar de un cBot propio.

6.3 Ventajas y riesgos frente al uso de un proveedor externo

- **Ventaja**: cero costos recurrentes por cuenta conectada (no hay factura de un tercero que escale con el número de usuarios).
- **Ventaja**: no se maneja ninguna credencial del bróker — se reduce la superficie de seguridad del proyecto.
- **Ventaja**: control total del código y posibilidad de agregar métricas propias sin esperar a un proveedor.
- **Riesgo — integridad de los datos**: al ejecutarse en el equipo del propio usuario, en teoría el reporte podría manipularse (a diferencia de leer directamente del servidor del bróker). Mitigación: compilar el EA a binario cerrado (.ex5/.ex4, no fuente), validar rangos y consistencia de los datos en el backend (ej. saltos de equity sin operación asociada), y registrar todo reporte con marca de tiempo y hash de integridad del payload.
- **Riesgo — dependencia de que el terminal esté abierto**: si el usuario cierra MetaTrader o pierde internet, el EA deja de reportar. Mitigación: campo `ea_last_report_at` por desafío, y una notificación automática si no hay reporte en más de N minutos ('cuenta desconectada').

- **Riesgo — onboarding más técnico:** instalar un EA y habilitar WebRequest agrega un paso manual frente a simplemente pegar una contraseña investor. Mitigación: video/wizard paso a paso dentro del flujo de vinculación (paso 4).

6.4 Integración con exchanges cripto

A diferencia de MT4/MT5, los exchanges centralizados (Binance, Bybit, OKX, etc.) sí ofrecen APIs REST/WebSocket oficiales y documentadas, por lo que aquí no se necesita un EA: el propio backend puede consultar la cuenta directamente.

- El usuario genera en el exchange una API key con permisos de **solo lectura** (sin trading, sin retiro, sin transferencias) y la pega en la plataforma.
- Según la firma cripto, la prueba puede correr sobre una cuenta **testnet** (ej. Binance Futures Testnet) o sobre una subcuenta real con capital simulado/limitado — esto se debe confirmar por cada firma que se cargue en el panel admin.
- El backend usa **CCXT** para normalizar la lectura de balance, posiciones e historial entre distintos exchanges sin escribir un cliente distinto por cada uno.
- El mismo motor de reglas (drawdown diario, drawdown máximo, objetivo, días mínimos) aplica igual que para Forex/Futuros — el preset de la firma cripto simplemente se carga con sus propios valores.

Motor de cálculo de reglas (independiente del origen del dato)

- **Drawdown diario:** se calcula sobre el balance/equity al inicio del día de servidor del bróker (reset a la hora que defina cada firma, normalmente medianoche del bróker).
- **Drawdown máximo:** estático (desde el balance inicial) o trailing (desde el máximo histórico / high water mark), según lo defina el preset de cada firma.
- **Objetivo de ganancia:** comparación de balance o equity actual contra el balance inicial + % objetivo del preset.
- **Días mínimos de trading:** se cuentan los días calendario en los que hubo al menos una operación cerrada.
- El motor debe correr como un job independiente y testeable, para poder simular escenarios y hacer pruebas unitarias exhaustivas antes de producción.

7. Sistema de notificaciones

Evento	Canal(es)	Disparador
Prueba aprobada	Email + in-app	Se cumple objetivo de ganancia y días mínimos, sin violar reglas
Prueba reprobada	Email + in-app	Se rompe el drawdown máximo o diario, o alguna regla especial
Alerta de DD diario	Email + in-app (push en fase 2)	El DD diario alcanza un umbral configurable, ej. 80% del límite
Cambio de fase	In-app	El usuario pasa de Fase 1 a Fase 2 en firmas de 2 pasos
Cuenta desconectada	Email	El worker pierde la conexión con la cuenta por más de X minutos

8. Hoja de ruta / fases de entrega

Fase	Duración est.	Entregables
0. Descubrimiento y arquitectura	1 semana	Definición final de reglas por firma, diagramas de arquitectura, elección definitiva de proveedor de datos (MetaApi/cTrader), setup de repos
1. Infraestructura base	1–2 semanas	Proyecto Supabase, esquema de base de datos, Auth, RLS, proyecto Next.js, CI/CD básico
2. Panel admin de firmas y presets	2 semanas	CRUD de firmas y presets, versionado de reglas, carga inicial de FTMO / Bullfy / FX Live Capital / Upcomers / FundedNext / The5ers / FundingPips (Forex) y Apex / Lucid / Upcomers / Topstep / MyFundedFutures (Futuros)
3. Selector de prueba (flujo usuario)	1–2 semanas	Pantallas de selección Forex/Futuros → firma → preset, formulario de vinculación de cuenta demo
4. EA/cBot propio + conector cripto	4–5 semanas	EA en MQL5/MQL4, validación de WebRequest, cBot en cAlgo (o integración con Open API de cTrader), conector cripto con CCXT (lectura de exchanges vía API key de solo lectura), endpoint receptor y almacenamiento de snapshots — fase de mayor riesgo técnico
5. Motor de reglas	2 semanas	Cálculo de DD diario, DD máximo, objetivo, días mínimos; pruebas unitarias con casos reales de cada firma
6. Dashboard en vivo	2 semanas	UI de métricas en tiempo real, gráficos de equity/drawdown, estado de la prueba, historial de operaciones
7. Notificaciones	1–2 semanas	Integración de Resend/SendGrid, plantillas de correo, eventos in-app vía Supabase Realtime
8. QA, seguridad y hardening	1–2 semanas	Auditoría de cifrado de credenciales, pruebas E2E, pruebas de carga del worker, revisión de RLS
9. Despliegue y lanzamiento	1 semana	Despliegue a producción, monitoreo con Sentry, documentación de operación

Duración total estimada: 15–19 semanas con un equipo de 3–5 personas trabajando en paralelo en frontend, backend/worker y QA.

9. Equipo recomendado

Rol	Dedicación	Enfoque principal
Tech Lead / Arquitecto	Tiempo completo	Arquitectura, decisiones técnicas, revisión de código
Desarrollador Backend/Worker	1–2, tiempo completo	Supabase, worker de MetaApi/cTrader, motor de reglas
Desarrollador Frontend	1–2, tiempo completo	Next.js, dashboard, panel admin
QA	Medio tiempo / por fases	Pruebas del motor de reglas y flujos críticos
DevOps (opcional)	Medio tiempo	CI/CD, monitoreo, escalado del worker

10. Riesgos y consideraciones

- **Integridad del reporte:** al depender de un EA que corre en el equipo del usuario, existe el riesgo teórico de manipulación de los datos reportados — ver mitigaciones en la sección 6.3 (binario compilado, validaciones de consistencia en backend, hash de payload).
- **Terminal debe permanecer abierto:** si el usuario cierra MetaTrader/cTrader o se queda sin internet, el EA deja de reportar — se debe distinguir claramente entre 'cuenta reprobada' y 'cuenta desconectada' para no generar falsos positivos.
- **Fricción de onboarding:** instalar el EA y habilitar WebRequest es más técnico que solo pegar una contraseña — requiere un tutorial claro dentro del producto para no perder usuarios en ese paso.
- **Compatibilidad cTrader:** debe confirmarse en Fase 0 si el sandbox de cAlgo permite peticiones HTTP salientes desde un cBot; si no, se usa la Open API oficial de cTrader como alternativa nativa.
- **Definición de firmas cripto:** el listado inicial del cliente no especificó firmas concretas de cripto (a diferencia de Forex/Futuros) — se debe investigar y confirmar cuáles firmas cripto se van a soportar y sobre qué exchange/testnet operan antes de cerrar los presets.
- **Testnet vs. capital real limitado:** algunas firmas cripto no ofrecen un modo 'demo' equivalente al de MT4/5 — puede requerir aclarar con cada firma si la evaluación corre en testnet o en una cuenta real con capital acotado.
- **Zona horaria del bróker:** el reseteo del drawdown diario depende de la hora del servidor del bróker, que varía entre brokers — el motor de reglas debe soportar esto por cuenta, no de forma global.
- **Diferencias entre firmas:** algunas prop firms usan drawdown 'trailing' y otras 'estático'; el modelo de presets ya contempla ambos casos, pero se debe validar caso por caso al cargar cada firma.

11. Próximos pasos inmediatos

- Confirmar y documentar las reglas exactas (objetivo, DD diario, DD máximo, días mínimos) de cada preset de FTMO, Bullfy, FX Live Capital, Upcomers, FundedNext, The5ers y FundingPips (Forex), y de Apex, Lucid, Upcomers, Topstep y MyFundedFutures (Futuros).
- Investigar y definir qué firmas cripto se van a soportar en el MVP, y sobre qué exchange(s) y modalidad (testnet o cuenta real limitada) evalúan a sus traders.
- Definir el protocolo de comunicación EA/cBot → backend (formato del JSON, frecuencia de envío, manejo de reintentos) y validar en Fase 0 si cAlgo permite peticiones HTTP salientes desde un cBot.

- Crear el proyecto en Supabase y definir el esquema inicial descrito en la sección 5.
- Definir el repositorio (monorepo recomendado: apps/web, apps/worker, packages/shared) y el pipeline de CI/CD.

Documento preparado como hoja de ruta técnica para ser entregado al equipo de desarrollo. Los tiempos son estimaciones y deben ajustarse tras la fase de descubrimiento (Fase 0).